

UNIVERSIDAD MAYOR DE “SAN ANDRÉS”

Facultad: Ciencias Puras y Naturales



PRESENTACION DE TRABAJO PRACTICA INDIVIDUALES

UNIVERSITARIO: JOAN JEREMY GONZALES ACARAPI

CARRERA: INFORMÁTICA

DOCENTE: LIC. MOISES SILVA

MATERIA: DESARROLLO DE APLICACIONES MULTIMEDIA

FECHA: 07/06/2026

LA PAZ-BOLIVIA

Actividades Individuales

A) Clasificación de Texturas

1. Descripción General

La aplicación implementa un sistema de clasificación automática de texturas en imágenes digitales. El algoritmo analiza cada píxel individualmente y lo asigna a una de cuatro categorías: Césped, Tierra, Cemento o Asfalto. Este enfoque es equivalente al utilizado en sistemas de clasificación de imágenes satelitales.

2. Metodología de Clasificación

2.1 Conversión al espacio de color HSV

Cada píxel RGB se convierte al espacio HSV (Hue, Saturation, Value). Esta conversión permite separar el tono cromático de la luminosidad, mejorando la robustez ante variaciones de iluminación.

2.2 Varianza Local 3x3

Para cada píxel se calcula la varianza de intensidad en su vecindario 3x3. Superficies rugosas (césped, tierra) presentan varianzas altas; superficies homogéneas (asfalto, cemento) presentan varianzas bajas.

2.3 Clasificación por Puntaje Combinado

Cada píxel recibe puntajes según dos criterios: (1) si sus valores H, S y V caen dentro de los rangos calibrados para cada clase, y (2) si su varianza local es consistente con la textura esperada. La clase con mayor puntaje acumulado (mínimo 2 puntos) es asignada.

3. Herramientas y Tecnologías

Tecnología	Uso en el proyecto
Python 3.9+	Lenguaje de programación principal
Pillow (PIL)	Carga, redimensión y exportación de imágenes
NumPy	Operaciones matriciales y cálculo de varianza local
Tkinter	Interfaz gráfica de escritorio (GUI)
threading	Procesamiento en hilo secundario para mantener UI responsiva

4. Mapa de Colores del Resultado

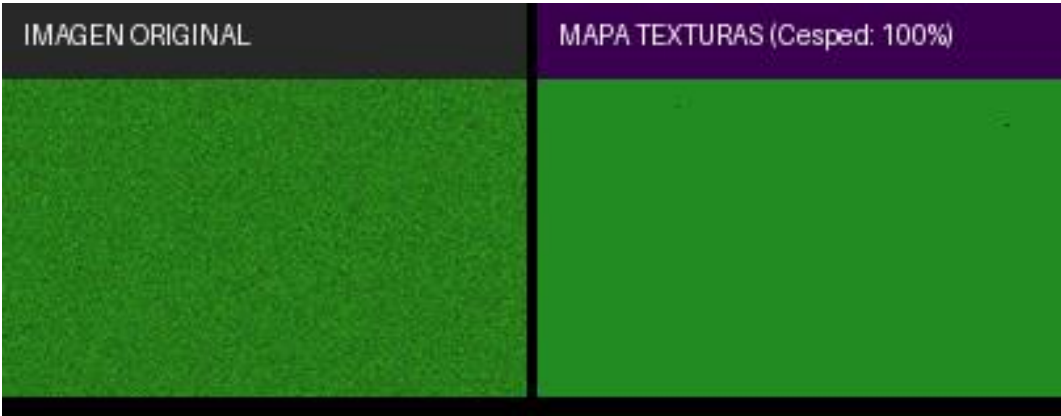
Clase	Color	RGB	Superficies típicas
Césped	Verde	(34, 139, 34)	Pasto, jardines, parques

Tierra	Marrón	(139, 90, 43)	Suelo descubierto, caminos
Cemento	Gris	(180, 180, 180)	Pavimento claro, concreto
Asfalto	Negro	(50, 50, 50)	Carreteras, pavimento oscuro
Otro	Morado	(190, 90, 190)	Píxeles sin clasificación clara

5. Comparación Visual: Original vs. Mapa de Texturas

A continuación, se muestran los resultados obtenidos al clasificar las imágenes de ejemplo. Izquierda: fotografía original. Derecha: mapa de texturas generado píxel a píxel.

Ejemplo 1 – Superficie de Césped



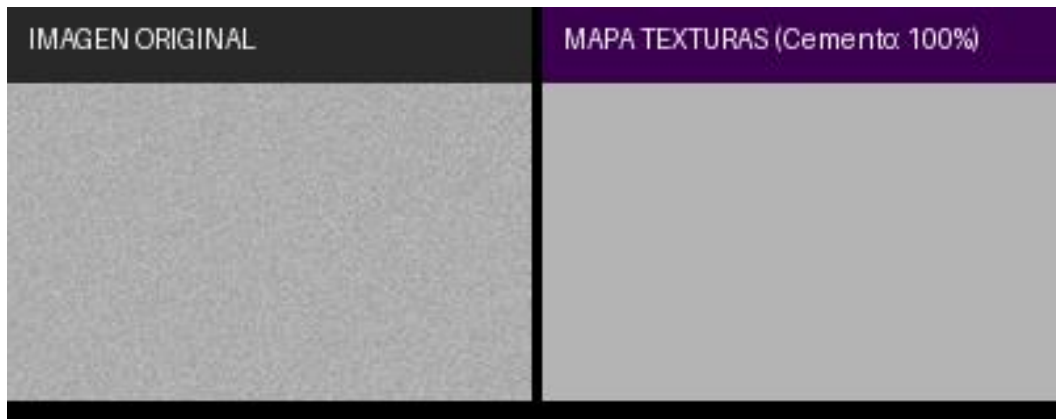
Izq.: imagen original | Der.: mapa de texturas clasificado píxel a píxel

Ejemplo 2 – Superficie de Tierra



Izq.: imagen original | Der.: mapa de texturas clasificado píxel a píxel

Ejemplo 3 – Superficie de Cemento



Izq.: imagen original | Der.: mapa de texturas clasificado píxel a píxel

Ejemplo 4 – Superficie de Asfalto



Izq.: imagen original | Der.: mapa de texturas clasificado píxel a píxel

6. Conclusiones

El clasificador demuestra cómo el análisis combinado de información cromática (HSV) y estructural (varianza local) permite diferenciar tipos de superficie eficazmente. La arquitectura modular facilita extender el sistema con nuevas clases definiendo sus rangos HSV en `clasificador.py` sin modificar la interfaz gráfica.

B) – Filtro de Suavizado 3×3

1. Descripción General

La aplicación implementa un filtro de promedio con ventana 3×3 aplicado directamente a nivel de píxel. El objetivo es reducir el ruido presente en imágenes digitales suavizando las transiciones abruptas de color. El programa permite aplicar el filtro de forma iterativa (1, 2 o 3 pasadas)

y visualizar simultáneamente la imagen original, la suavizada y la imagen de diferencia amplificada.

2. Metodología del Filtro

2.1 Filtro de Promedio (Box Filter)

Para cada píxel (i, j) , su nuevo valor se calcula como el promedio de los 9 píxeles en su vecindario 3×3 , aplicado de forma independiente en cada canal R, G, B:

$$\text{pixel_nuevo}[i,j] = (1/9) \times \sum \text{pixel}[i+di, j+dj] \quad \text{para } di, dj \in \{-1, 0, 1\}$$

2.2 Manejo de Bordes – Padding Espejo

Los píxeles en los bordes se extienden especularmente (reflect padding) antes de aplicar el filtro, evitando artefactos en los extremos de la imagen.

2.3 Aplicación en Múltiples Pasadas

Cada pasada toma como entrada el resultado de la anterior. Aplicar el filtro 3×3 tres veces consecutivas es matemáticamente equivalente a una aproximación del filtro gaussiano, produciendo un suavizado más natural y progresivo.

3. Herramientas y Tecnologías

Tecnología	Uso en el proyecto
Python 3.9+	Lenguaje de programación principal
Pillow (PIL)	Carga, conversión y exportación de imágenes
NumPy	Operaciones matriciales, padding espejo y métricas
Tkinter	Interfaz gráfica de escritorio
threading	Procesamiento paralelo para mantener la UI responsiva

4. Métricas de Calidad Implementadas

Métrica	Descripción
MSE – Error Cuadrático Medio	Magnitud promedio del cambio: $\text{mean}((\text{original} - \text{suavizada})^2)$
MAE – Error Absoluto Medio	Promedio de cambio por píxel: $\text{mean}(\text{original} - \text{suavizada})$
PSNR – Pico Señal-Ruido	Calidad percibida en dB = $10 \cdot \log_{10}(255^2/\text{MSE})$. Valores >30 dB indican buena calidad
Píxeles modificados (%)	Porcentaje de píxeles cuyo cambio supera 5 niveles de intensidad

5. Comparación Visual: Antes vs. Después del Filtro

Las imágenes muestran de izquierda a derecha: (1) original con ruido, (2) suavizado 1 pasada, (3) suavizado 3 pasadas, (4) diferencia amplificada $\times 5$ – visualiza el ruido eliminado.

Ejemplo 1 – Imagen de Rostro Sintético con Ruido



[Original] → [Suavizado 1 pasada] → [Suavizado 3 pasadas] → [Diferencia $\times 5$]

Ejemplo 2 – Imagen de Ciudad con Ruido



[Original] → [Suavizado 1 pasada] → [Suavizado 3 pasadas] → [Diferencia $\times 5$]

6. Resultados Numéricos Obtenidos

Imagen / Nro. de Pasadas	MSE	PSNR (dB)	Píxeles Mod. (%)
Rostro – 1 pasada	81.2	29.0	44.0 %
Rostro – 3 pasadas	130.0	27.0	48.9 %
Ciudad – 1 pasada	145.6	26.5	57.0 %
Ciudad – 3 pasadas	248.2	24.2	63.6 %

Interpretación: A mayor número de pasadas, mayor reducción de ruido pero mayor pérdida de detalle (bordes difuminados). El PSNR >26 dB en todos los casos indica que la imagen suavizada conserva calidad visual aceptable.

7. Conclusiones

El filtro de promedio 3×3 demuestra el principio fundamental del procesamiento de señales: el suavizado espacial reduce ruido a costa de

pérdida de detalle. Con 1 pasada se elimina ruido puntual preservando bordes; con 3 pasadas el suavizado es más intenso pero los bordes se difuminan. La imagen de diferencia amplificada revela con precisión qué información fue modificada por el filtro.

C) – Cover Multimedia: La Vaca Lola

1. Descripción General

El proyecto consiste en una producción multimedia de la canción infantil "La Vaca Lola", adoptando la identidad visual del artista urbano Bad Bunny / J Balvin, y vinculando el audio con una animación donde el ritmo y el movimiento coinciden de forma sincronizada. Se desarrolló una aplicación de escritorio en Python utilizando Pygame como motor de renderizado, integrando animación generativa, reproducción de audio sincronizada, y un video generado con Inteligencia Artificial.

2. Herramientas de Inteligencia Artificial Utilizadas

Herramienta IA	Uso específico en el proyecto
ChatGPT (OpenAI)	Generación de imagen base del personaje principal (artista)
Wav2Lip (wav2lip.org)	Sincronización labial: imagen + audio → video MP4 de 10 segundos
Suno AI	Generación y estructuración de la letra con marcas de tiempo

3. Herramientas y Tecnologías de Desarrollo

Tecnología	Uso en el proyecto
Python 3.10+	Lenguaje principal de la aplicación
Pygame	Motor de renderizado, animación y reproducción de audio a 60 FPS
OpenCV (cv2)	Decodificación y reproducción de video MP4 frame a frame
NumPy	Conversión de frames BGR a RGB para Pygame
ChatGPT	Generación de imagen base del personaje
Wav2Lip	Video con sincronía labial a partir de imagen + audio
Suno AI	Letra musical con estructura temporal (timestamps)

4. Metodología de Desarrollo

El proyecto siguió una metodología de producción multimedia en cuatro etapas:

- Etapa 1 – Generación de contenido IA: Se usó ChatGPT para crear la imagen base del personaje, Wav2Lip para convertirla en video con sincronía labial, y Suno AI para generar la letra con marcas de tiempo.
- Etapa 2 – Motor de animación: Se implementó el bucle principal en Pygame a 60 FPS con personajes animados (monigotes, vaca) que responden al beat de la música mediante una variable de intensidad calculada a partir del BPM.
- Etapa 3 – Integración de audio y video: Se vinculó el audio con pygame.mixer, la letra mediante timestamps en letras.py comparados con get_pos(), y el video MP4 (bucle de 10 seg) decodificado frame a frame con OpenCV.
- Etapa 4 – Control de pausa global: Al presionar espacio, se detienen simultáneamente el audio, la animación y el video, reanudando exactamente desde el mismo punto.

5. Arquitectura del Software

Módulo / Archivo	Responsabilidad
main.py	Punto de entrada de la aplicación
config.py	Constantes globales: resolución, colores, FPS, BPM
src/escena.py	Controlador principal y bucle de juego
src/personajes/monigote.py	Stick-figure humano animado (Bad Bunny / J Balvin)
src/personajes/vaca.py	Vaca animada con bounce y cola oscilante
src/audio/gestor_audio.py	Carga y control de reproducción con pygame.mixer
src/audio/letras.py	Letra con timestamps en segundos para sincronización
src/assets/video/video.mp4	Video generado con Wav2Lip (bucle de 10 seg)
src/ui/pantalla_show.py	Renderizado principal con video integrado
src/ui/hud.py	HUD: barra de beat, estado y controles
src/ui/particulas.py	Sistema de partículas decorativas
src/utils/anim.py	Funciones de animación (rebote, pulso)

6. Controles de la Aplicación

Tecla	Acción
ESPACIO	Pausa / Reanuda todo (audio + animación + video simultáneamente)
R	Volver al menú principal
ESC	Salir de la aplicación

7. Conclusiones

Este proyecto demuestra la integración efectiva de múltiples tecnologías multimedia en un único entorno interactivo. La combinación de animación generativa en tiempo real, sincronización precisa de audio y letra, y la incorporación de un video generado con Inteligencia Artificial logran una experiencia audiovisual coherente y dinámica. El uso de ChatGPT, Wav2Lip y Suno AI simplifica la producción de contenido multimedia y abre nuevas posibilidades creativas, permitiendo generar personajes animados cantantes a partir de una simple imagen y un fragmento de audio.